

LAPORAN PRAKTIKUM KOMPREHENSIF
TUGAS 1 OPERASI SYSTEM



Dosen Pengampu :
Triawan Adi Cahyanto M.kom

Disusun oleh:
HAFABI ARROHIM (2410651013) (A)

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
TAHUN 2025

1. Tujuan Praktikum

Tujuan dari praktikum ini adalah untuk memahami konsep dasar manajemen proses dan komunikasi antar proses (Interprocess Communication / IPC) di sistem operasi Linux dengan menggunakan system call level rendah seperti fork(), wait(), shmget(), shmat(), dan shmdt(). Mahasiswa diharapkan mampu menjelaskan bagaimana proses parent dan child berinteraksi melalui shared memory serta mengelola eksekusi proses secara sinkron

2. Dasar Teori

~ proses

Proses adalah program yang sedang di eksekusi. Setiap proses memiliki ruang memori sendiri dan dijalankan secara independent.

~system call

System call adalah mekanisme untuk mengakses layanan kernel dari program user. Contoh system call yang di gunakan pada percobaan ini antara lain:

- Fork() : membuat proses anak (child proses)
- Wait() : menunggu proses anak selesai
- Shmget() : membuat segmen shared memory
- Shmat() : menempelkan (attach) shared memory ke ruang alamat proses
- Shmdt() : melepaskan shared memory dari proses
- Shmctl() : menghapus segmen shared memory

~shared memory

Merupakan salah satu metode komunikasi antar proses (IPC). Shared memory memungkinkan dua atau lebih proses untuk mengakses area memori yang sama sehingga dapat saling bertukar data dengan cepat.

~ Interprocess Communication (IPC)

IPC memungkinkan beberapa proses berkoordinasi dan bertukar data, baik secara sinkron (bergantian) maupun asinkron (bersamaan).

3. Deskripsi Program

Program ini ditulis dalam bahasa C dan dijalankan pada sistem operasi Linux (Ubuntu).

Program membuat shared memory menggunakan system call `shmget()` dan membaginya antara proses parent dan child. Langkah-langkah utama dalam program :

- ~Membuat shared memory
- ~Melakukan untuk membuat proses anak.
- ~proses parent menulis data ke shared memory
- ~proses child membaca data dari shared memory.
- ~shared memory di lepaskan (`shmdt`) dan dihapus (`shmctl`) setelah selesai.

Karena NIM berakhiran 3 (ganjil), maka program menggunakan fitur tambahan X, misalnya:

Menambahkan proses tambahan (child kedua) yang menghitung panjang data dari shared memory dan mencetak hasilnya ke layar.

4. Flowchart Program

- ~Mulai
- ~Buat shared memory
- ~Lakukan `fork()`
- ~Cek proses:
- ~Jika child, jalankan tugas (faktorial / prima / balik string)
- ~Jika parent, tunggu proses anak selesai
- ~Tampilkan hasil output
- ~Hapus shared memory
- ~Selesai

KODE SOURCE

```
GNU nano 7.2 process_manager.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <sys/ipc.h>
#include <sys/shm.h>

/* =====
Fungsi bantu
===== */

// Fungsi untuk membalik string (karena strrev tidak tersedia di Linux)
void balik_string(char *str) {
    int len = strlen(str);
    for (int i = 0; i < len / 2; i++) {
        char tmp = str[i];
        str[i] = str[len - i - 1];
        str[len - i - 1] = tmp;
    }
}

// Hitung faktorial
long long faktorial(int n) {
    long long res = 1;
    for (int i = 1; i <= n; i++) res *= i;
    return res;
}

// Hitung prima, hasil ditulis ke output (pastikan output punya cukup ukuran)
```

```
GNU nano 7.2
/* =====
Program utama
===== */

int main() {
    int pipefd[2][2];          // pipe untuk child1 & child2
    pid_t pid[3];
    key_t key = 1234;
    int shmid = shmget(key, 1024, IPC_CREAT | 0666);
    if (shmid < 0) {
        perror("shmget");
        exit(1);
    }
    char *shared_mem = (char *)shmat(shmid, NULL, 0);
    if (shared_mem == (char *)-1) {
        perror("shmat");
        exit(1);
    }

    // buat 2 pipe (child1 & child2)
    for (int i = 0; i < 2; i++) {
        if (pipe(pipefd[i]) == -1) {
            perror("pipe");
            exit(1);
        }
    }

    for (int i = 0; i < 3; i++) {
        pid[i] = fork();
        if (pid[i] < 0) {
            perror("fork");
        }
    }
}
```

```
GNU nano 7.2 process_manager.c

if (pid[i] == 0) { // CHILD
    struct timeval start, end;
    gettimeofday(&start, NULL);

    if (i == 0) { // Child 1: faktorial via pipe
        close(pipefd[0][0]); // tutup read end
        int n;
        printf("Child 1: Masukkan angka untuk faktorial: ");
        scanf("%d", &n);
        gettimeofday(&start, NULL); // mulai waktu setelah input
        long long res = faktorial(n);
        write(pipefd[0][1], &res, sizeof(res));
        close(pipefd[0][1]);
    }
    else if (i == 1) { // Child 2: prima via pipe
        close(pipefd[1][0]);
        int batas;
        printf("Child 2: Masukkan batas bilangan prima: ");
        scanf("%d", &batas);
        gettimeofday(&start, NULL);
        char primes[512];
        hitung_prima(batas, primes, sizeof(primes));
        write(pipefd[1][1], primes, strlen(primes) + 1);
        close(pipefd[1][1]);
    }
    else { // Child 3: pakai shared memory
        char str[200];
        printf("Child 3: Masukkan string: ");
        scanf("%199s", str); // batasi input
        gettimeofday(&start, NULL);
    }
}
```

```

// exit dengan kode unik (boleh i+1)
exit(i + 1);
}
// parent loop lanjut membuat child selanjutnya
}

// PARENT: tutup sisi tulis pipe, nanti baca
for (int i = 0; i < 2; i++) close(pipefd[i][1]);

// tunggu dan baca hasil tiap child
for (int i = 0; i < 3; i++) {
    int status;
    pid_t w = waitpid(pid[i], &status, 0);
    if (w == -1) perror("waitpid");
    printf("\n[Parent] Child PID: %d selesai. Exit code: %d\n", (int)w, WIFEXITED(status) ? WEXITSTATUS(status) : -1);

    if (i == 0) {
        long long res;
        read(pipefd[0][0], &res, sizeof(res));
        printf("Hasil faktorial: %lld\n", res);
        close(pipefd[0][0]);
    } else if (i == 1) {
        char primes[512] = {0};
        read(pipefd[1][0], primes, sizeof(primes));
        printf("Bilangan prima: %s\n", primes);
        close(pipefd[1][0]);
    } else {
        printf("Hasil dari shared memory: %s\n", shared_mem);
    }
}
}

```

OUTPUT YANG DI HASILKAN

```

faby@fabyar-biee:~/TugasOS$ nano process_manager.c
faby@fabyar-biee:~/TugasOS$ ./process_manager
Child 1: Masukkan angka untuk faktorial: Child 2: Masukkan batas bilangan prima: Child 3: Masukkan string: 8
[Child 1] Selesai dalam 0.000025 detik

[Parent] Child PID: 845 selesai. Exit code: 1
Hasil faktorial: 40320
10
[Child 2] Selesai dalam 0.000039 detik

[Parent] Child PID: 846 selesai. Exit code: 2
Bilangan prima: 2 3 5 7
rohim
[Child 3] Selesai dalam 0.000053 detik

[Parent] Child PID: 847 selesai. Exit code: 3
Hasil dari shared memory: String dibalik: mihor
faby@fabyar-biee:~/TugasOS$ |

```

5. Kesimpulan

Dari hasil praktikum ini, dapat disimpulkan bahwa sistem operasi Linux menyediakan mekanisme komunikasi antar proses (IPC) yang efisien melalui shared memory. Dengan system call seperti `fork()`, `shmget()`, dan `shmat()`, proses parent dan child dapat saling bertukar data secara langsung tanpa file perantara. Selain itu, melalui implementasi manual di level low-level, mahasiswa memahami bagaimana kernel berperan dalam manajemen proses dan memori bersama